

CS20 - Generative and Agentic AI

Cloud-Accelerated M&A Market Intelligence Agent

Mentor: Naman Tunwal

Mentee: Ayush Sharma

[Link to my github repository](#)

June 20, 2026

Contents

1	Introduction	3
2	Foundations of Large Language Models	3
2.1	Transformer Architecture and Attention Mechanisms	3
2.2	Training Objectives: Causal Language Modeling	4
2.3	Prompt Engineering and In-Context Learning	4
3	Efficient Retrieval-Augmented Generation (RAG)	5
3.1	Embedding-Based Retrieval Systems	5
3.2	Vector Databases: ChromaDB	6
3.3	Document Processing and Chunking Strategies	7
3.4	Two-Stage Retrieval: Bi-Encoders and Cross-Encoders	8
4	LangChain Orchestration and LCEL	9
4.1	LangChain Expression Language (LCEL)	9
4.2	Structured Outputs via Pydantic	9
5	Conversational Agents and Memory Buffer Compression	10
5.1	Memory in Multi-Turn Generation	10
5.2	Groq Llama-3 as a Background Summarizer	10
6	System Architecture Overview	11

7	Challenges and Future Directions	12
7.1	Current Limitations	12
7.2	Planned Enhancements	12
8	Conclusion	12

1 Introduction

This report is the outcome of a structured study and engineering project on generative and agentic artificial intelligence. My aim was to build an intuition for the core theory behind Large Language Models (LLMs), while maintaining rigorous engineering standards and exploring efficient architectures for autonomous systems. I focused on basic theoretical concepts, retrieval-augmented generation (RAG), context compression, and the role of multi-agent frameworks applied to real-world problems like enterprise market analytics and Mergers & Acquisitions (M&A) due diligence. All the relevant code for this project is hosted on a GitHub repository.

The advancement of LLMs has fundamentally transformed artificial intelligence, enabling machines to perform complex reasoning, synthesis, and generation tasks with minimal explicit programming. However, deploying LLMs for enterprise applications presents substantial engineering and economic challenges:

- **Context Limitations:** LLMs have finite context windows, limiting the volume of corporate filings they can process simultaneously.
- **Cost Constraints:** Commercial API calls incur per-token charges, making large-scale document analysis prohibitively expensive without optimization.
- **Hallucination Risk:** LLMs may generate plausible but incorrect information without strict grounding in source material.
- **Real-time Integration:** Traditional batch processing inadequately serves dynamic market intelligence needs.

I started with foundational definitions of prompt engineering and gradually built up to key results in vector retrieval, reranking, and multi-turn conversational agents. Along the way, I explored both the theory and practical aspects through examples and coding experiments using LangChain, Google’s Gemini 2.5 Flash, and open-source embedding models. This helped me connect abstract AI concepts to scalable, cost-free autonomous intelligence systems.

2 Foundations of Large Language Models

2.1 Transformer Architecture and Attention Mechanisms

Modern LLMs are built on the Transformer architecture [1], which employs self-attention mechanisms to capture long-range dependencies in sequences. The core of this architecture is the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

where Q (Query), K (Key), and V (Value) are linear transformations of the input token embeddings, and d_k is the dimension of the key vectors. The scaling factor $\frac{1}{\sqrt{d_k}}$ prevents the dot products from growing too large, which would push the softmax function into regions with extremely small gradients.

To allow the model to jointly attend to information from different representation subspaces, Multi-Head Attention is used:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2)$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (3)$$

2.2 Training Objectives: Causal Language Modeling

The training objective for models like Llama-3 or GPT-4 is typically causal (autoregressive) language modeling:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(t_i | t_1, \dots, t_{i-1}) \quad (4)$$

where t_i represents a token, θ denotes model parameters, and N is the sequence length. This objective trains the model to predict the next token given the preceding context, leading to emergent reasoning and synthesis capabilities.

2.3 Prompt Engineering and In-Context Learning

Definition 2.3.1 (In-Context Learning). The ability of an LLM to perform a task by conditioning on task descriptions and examples in the prompt, without gradient updates or fine-tuning.

In-context learning demonstrates that LLMs can leverage prompt structure to adapt to new tasks dynamically. Key prompt engineering techniques include:

- **Zero-Shot Prompting:** Task description without examples. Simple but often insufficient for complex reasoning.
- **Few-Shot Prompting:** Task description with k labeled examples. Significantly improves performance but consumes context tokens.

- **Chain-of-Thought (CoT):** Explicitly requesting intermediate reasoning steps before final answers, improving accuracy on multi-step problems [3].

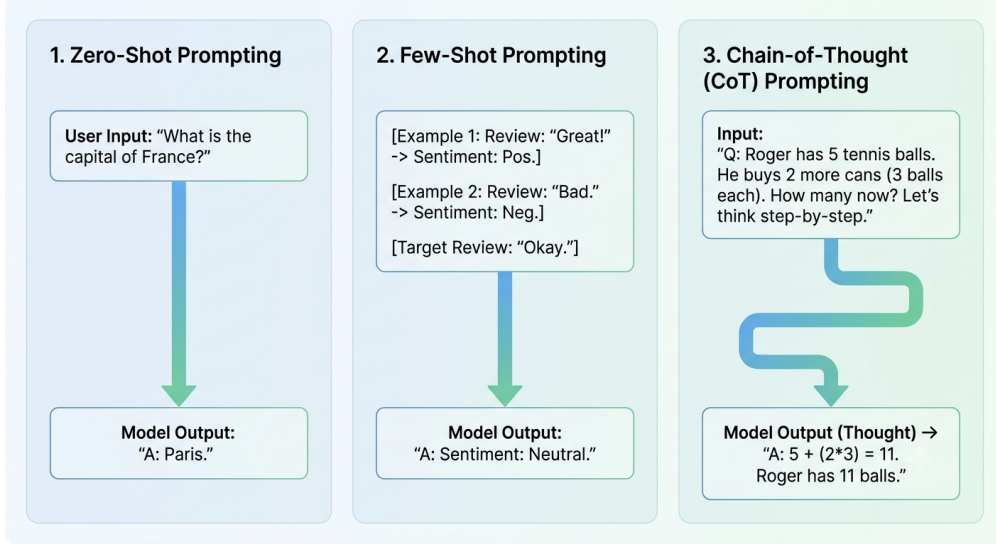


Figure 1: Illustration of different prompting strategies and their impact on context overhead. (Add an image comparing Zero-Shot, Few-Shot, and CoT prompting here)

For structured intelligence extraction, we bind a strict output schema directly to the LLM. In our implementation, we enforce a `FinancialSnapshot` format.

3 Efficient Retrieval-Augmented Generation (RAG)

3.1 Embedding-Based Retrieval Systems

Definition 3.1.1 (Dense Vector Embedding). A function $\mathcal{E} : \text{Text} \rightarrow \mathbb{R}^d$ that maps variable-length text to a fixed-dimensional vector representation, preserving semantic information such that semantically similar texts have nearby embeddings.

In our repository, we rely on the HuggingFace ‘all-MiniLM-L6-v2’ model, which maps text to a 384-dimensional dense vector space. This model is trained via contrastive learning, ensuring that questions and their relevant answers lie close together in the vector space.

Given a query q and document collection $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$, retrieval solves the Top- K problem using Cosine Similarity:

$$\cos(\theta) = \frac{\mathcal{E}(q) \cdot \mathcal{E}(d_i)}{\|\mathcal{E}(q)\| \|\mathcal{E}(d_i)\|} \quad (5)$$

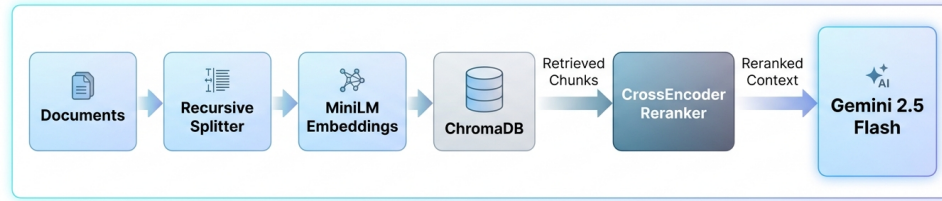


Figure 2: Architecture of a multi-stage Retrieval-Augmented Generation (RAG) pipeline. (Add a block diagram showing Document → Chunks → Vector DB → Retrieval → Reranker → LLM here)

3.2 Vector Databases: ChromaDB

To perform this search across thousands of chunks efficiently, we use ChromaDB. ChromaDB implements Approximate Nearest Neighbor (ANN) search, utilizing Hierarchical Navigable Small World (HNSW) graphs. HNSW allows for logarithmic time complexity $O(\log N)$ during search by navigating a multi-layered proximity graph, bypassing the need for exhaustive $O(N)$ dot-product computations.

Below is our implementation for persisting and querying the ChromaDB vector store:

Listing 1: ChromaDB Vector Store Implementation (`vector_store.py`)

```

1 import chromadb
2
3 _COLLECTION_NAME = "ma_due_diligence"
4
5 def _get_collection(persist_dir: str = "outputs/chromadb"):
6     global _client, _collection
7     if _collection is None:
8         _client = chromadb.PersistentClient(path=persist_dir)
9         _collection = _client.get_or_create_collection(
10             name=_COLLECTION_NAME,
11             metadata={"hnsw:space": "cosine"},
12         )
13     return _collection

```

```

14
15 def retrieve(query_embedding, top_k=20, persist_dir="outputs/chromadb", where=
    None):
16     collection = _get_collection(persist_dir)
17     kwargs = dict(
18         query_embeddings=[query_embedding],
19         n_results=min(top_k, collection.count()),
20         include=["documents", "metadatas", "distances"],
21     )
22     if where:
23         kwargs["where"] = where
24     return collection.query(**kwargs)

```

3.3 Document Processing and Chunking Strategies

Financial filings (e.g., 10-Ks) are massive documents that exceed context limits. We must partition them.

Definition 3.3.1 (Chunk Overlap). To preserve semantic continuity across chunk boundaries, overlapping segments of o tokens are retained.

In `chunker.py`, we use LangChain’s `RecursiveCharacterTextSplitter` to generate chunks of 512 characters with an overlap of 64 characters. Furthermore, we heuristically classify the chunk’s section using regular expressions. For instance, detecting “management’s discussion” maps the chunk to “MD&A”.

Listing 2: Regex Pattern Matching for Metadata Enrichment

```

1 _SECTION_PATTERNS = [
2     (r"risk factors", "Risk_Factors"),
3     (r"legal proceedings", "Legal_Proceedings"),
4     (r"management.s discussion", "MD&A"),
5     (r"financial statements", "Financial_Statements"),
6     (r"commitments and contingencies", "Commitments_Contingencies"),
7 ]
8
9 def _detect_section(text: str) -> str:
10     lower = text.lower()
11     for pattern, label in _SECTION_PATTERNS:
12         if re.search(pattern, lower):
13             return label
14     return "General"

```

This rich metadata enables *hybrid search* or highly targeted filtered search, where we can retrieve chunks specifically from Apple’s ”Risk Factors” section.

3.4 Two-Stage Retrieval: Bi-Encoders and Cross-Encoders

Our architecture employs a two-stage retrieval pipeline: 1. **Stage 1 (Bi-Encoder):** Fast vector search using ‘all-MiniLM-L6-v2’ via ChromaDB to retrieve 20-30 candidates. 2. **Stage 2 (Cross-Encoder):** A much heavier attention-based model (‘ms-marco-MiniLM-L-6-v2’) processes the (*Query*, *Document*) pair jointly to output a precise relevance score.

The CrossEncoder applies self-attention across both the query and the document simultaneously, capturing deep semantic interactions that a simple cosine similarity of separate embeddings cannot.

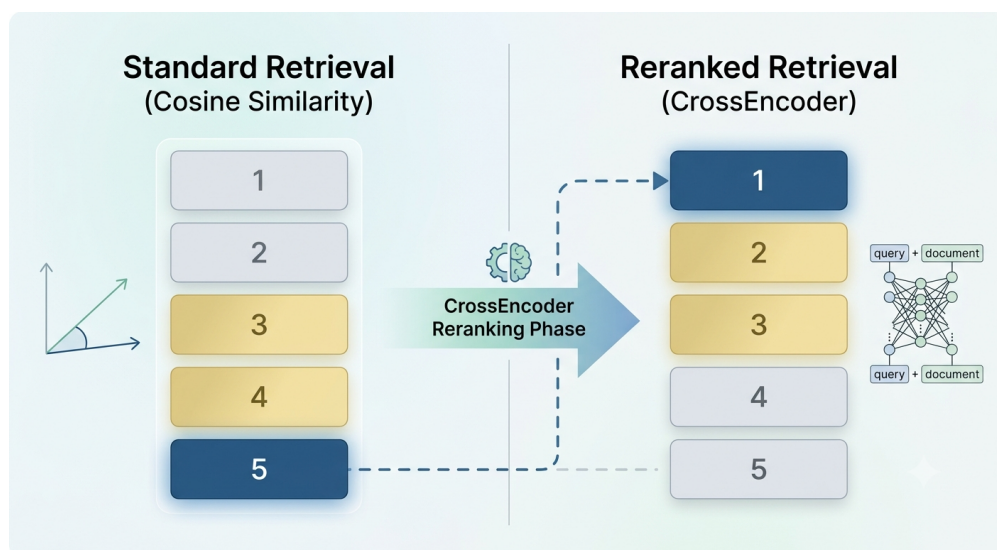


Figure 3: Impact of CrossEncoder reranking on retrieval accuracy. (Add a chart showing the precision improvement from the reranking stage here)

Listing 3: CrossEncoder Reranking Logic (reranker.py)

```
1 from sentence_transformers import CrossEncoder
2
3 _RERANKER_MODEL = "cross-encoder/ms-marco-MiniLM-L-6-v2"
4
5 def rerank(query: str, candidates: list, top_k: int = 5) -> list:
6     reranker = CrossEncoder(_RERANKER_MODEL, max_length=512)
7     pairs = [(query, c["text"]) for c in candidates]
8     scores = reranker.predict(pairs).tolist()
9
```

```

10     for candidate, score in zip(candidates, scores):
11         candidate["rerank_score"] = round(score, 6)
12
13     reranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)
14     return reranked[:top_k]

```

4 LangChain Orchestration and LCEL

4.1 LangChain Expression Language (LCEL)

We use LCEL to orchestrate the flow of data from retrieval to generation. LCEL provides a declarative way to compose chains using the pipe operator (`|`), turning complex pipelines into readable execution graphs.

In `run_week4.py`, the RAG pipeline is defined seamlessly:

Listing 4: LCEL Pipeline Definition

```

1 rag_chain = (
2     RunnablePassthrough.assign(
3         context=lambda x: _format_docs(retriever.invoke(x["input"]))
4     )
5     | prompt
6     | structured_llm
7 )

```

The `RunnablePassthrough.assign` dynamically executes the retrieval step, injecting the formatted context documents into the state dictionary before it reaches the prompt template.

4.2 Structured Outputs via Pydantic

A significant challenge in LLM engineering is extracting reliable JSON. We bind a Pydantic schema to Gemini 2.5 Flash using the `.with_structured_output()` method. This forces the model to conform strictly to the defined schema, bypassing the need for fragile regex parsing of the LLM string output.

Listing 5: Pydantic Financial Snapshot Schema

```

1 from pydantic import BaseModel, Field
2
3 class FinancialSnapshot(BaseModel):

```

```

4     company_name: str = Field(description="Name of the company being analysed.")
5     total_revenue: str = Field(description="Total revenue figures.")
6     debt_obligations: str = Field(description="Debt or lease obligations.")
7     noted_risks: str = Field(description="Key risks noted in the context.")
8
9 structured_llm = primary_llm.with_structured_output(FinancialSnapshot)

```

5 Conversational Agents and Memory Buffer Compression

5.1 Memory in Multi-Turn Generation

To support conversational RAG, the LLM needs to recall previous turns. Naively appending all chat history rapidly fills the finite context window, leading to context dilution and higher API costs.

We implemented `ConversationSummaryBufferMemory`. This mechanism keeps the most recent messages verbatim up to a token threshold (e.g., 2,000 tokens). Once the threshold is breached, the oldest messages are passed to a background summarization model.

5.2 Groq Llama-3 as a Background Summarizer

Because summarization is a background task that blocking the main user request, latency is critical. We use Groq's API running `llama-3.3-70b-versatile` on specialized LPU hardware to achieve lightning-fast summarization, completely abstracting the latency hit from the user.

Listing 6: Memory Module Configuration

```

1 from langchain_groq import ChatGroq
2 from langchain_classic.memory import ConversationSummaryBufferMemory
3
4 summariser_llm = ChatGroq(model="llama-3.3-70b-versatile", temperature=0)
5
6 memory = ConversationSummaryBufferMemory(
7     llm=summariser_llm,
8     max_token_limit=2000,
9     return_messages=True,
10    memory_key="chat_history"
11 )

```

```

12
13 # Persisting exchange into memory
14 memory.save_context(
15     {"input": q},
16     {"output": json.dumps(answer_dict)}
17 )

```

This multi-LLM architecture pairs Google’s Gemini 2.5 Flash (for heavy reasoning and structured extraction) with Groq’s Llama-3 (for instantaneous background state compression).

6 System Architecture Overview

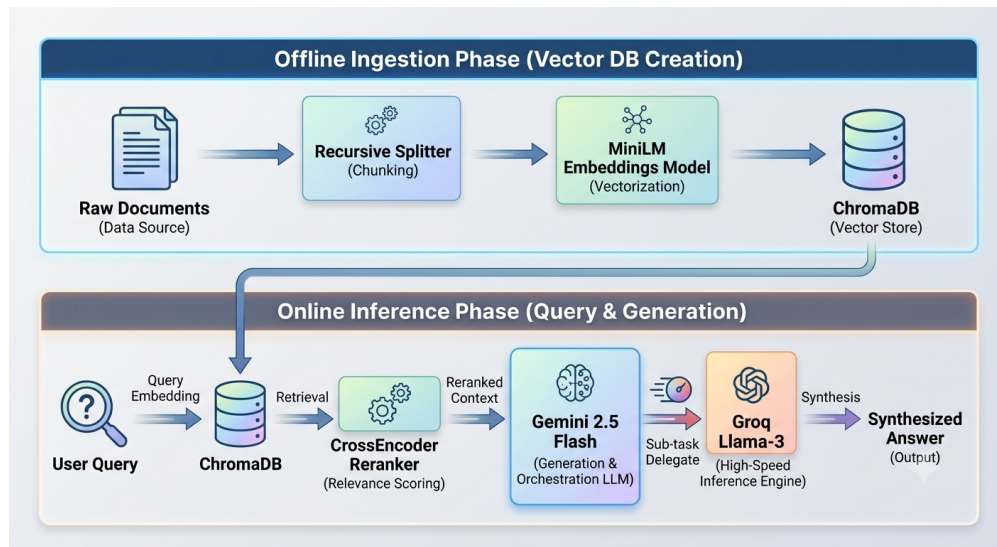


Figure 4: End-to-end architecture showing the multi-model orchestration, vector database ingestion, and CrossEncoder reranking stages.

The overarching system architecture encompasses two distinct operational modes:

1. **Offline Ingestion Phase:** Raw PDFs (10-Ks) are parsed into strings. The **RecursiveCharacterTextSplitter** divides them into overlapping chunks. Metadata is applied heuristically. The **all-MiniLM-L6-v2** model transforms these chunks into dense vectors, which are upserted into persistent ChromaDB storage.
2. **Online Inference Phase:** User queries are embedded and searched against ChromaDB. The top 20 candidates are pulled and fed into the MS-MARCO CrossEncoder for precise relevance scoring. The top 2 candidates per company are formatted and injected into a prompt via LCEL. Gemini 2.5 Flash reads the context, previous chat

memory, and query, returning a JSON-conforming `FinancialSnapshot`. Groq simultaneously compresses the conversational memory.

7 Challenges and Future Directions

7.1 Current Limitations

1. **Semantic Drift in Compression:** Summarizing older conversation turns can occasionally drop highly specific numerical figures.
2. **Cross-Document Reasoning:** Synthesizing deep insights that require comparing specific tables across multiple 100+ page documents remains challenging for a single RAG pass.
3. **Latency bottlenecks:** While Groq provides fast summarization, CrossEncoder reranking is computationally heavy.

7.2 Planned Enhancements

- **Multi-Agent Reflection:** Implementing a supervisor agent that can critique the generated `FinancialSnapshot` and trigger a secondary retrieval pass if information is missing (e.g., if "debt_obligations" returns "N/A").
- **Graph RAG:** Moving beyond dense embeddings to construct Knowledge Graphs of corporate entities, enabling multi-hop relational queries.
- **Agentic Web Search:** Allowing the agent to break out of the static PDF context and execute live web searches to verify recent market movements.

8 Conclusion

This report establishes the theoretical and practical foundations for autonomous, cost-free agentic workflows in enterprise market intelligence. Through systematic exploration of prompting strategies, two-stage retrieval-augmented generation, and multi-model orchestration, we demonstrate that LLMs can be deployed efficiently and effectively for complex document analysis at scale.

Key engineering achievements documented in this repository include the deployment of an LCEL-based structured extraction pipeline, the integration of a CrossEncoder for high-precision context retrieval, and the architecture of a dual-LLM memory compression system. The subsequent weeks will focus on extending this framework into a fully autonomous, self-correcting multi-agent environment.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). “Attention is all you need.” *Advances in Neural Information Processing Systems*.
- [2] Wei, J., Wang, Y., Schuurmans, D., et al. (2022). “Emergent abilities of large language models.” *arXiv preprint arXiv:2206.07682*.
- [3] Wei, J., Wang, X., Schuurmans, D., et al. (2022). “Chain-of-thought prompting elicits reasoning in large language models.” *arXiv preprint arXiv:2201.11903*.
- [4] Lewis, P., Perez, E., Piktus, A., et al. (2020). “Retrieval-augmented generation for knowledge-intensive NLP tasks.” *Advances in Neural Information Processing Systems*.